Industrial Internship Report on

”Password Manager”

Prepared by

[A H RAJADITYA]

Executive Summary

This report presents a comprehensive overview of the work completed during the 6-week industrial internship conducted by upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt Ltd (UCT). The internship aimed to provide hands-on experience in solving real-world problems using programming and analytical skills.

The project undertaken during this internship is **“Password Manager using Python”**, which focuses on designing and developing a secure system for storing, managing, and retrieving user credentials. In today’s digital environment, individuals manage multiple online accounts across various platforms, making password management a critical challenge. Many users tend to reuse weak passwords or store them in insecure formats, leading to increased vulnerability to cyber threats and data breaches.

The primary objective of this project is to develop a Python-based application that addresses these issues by providing a **secure, efficient, and user-friendly solution** for password management. The system is designed to store passwords in an encrypted or hashed format, ensuring that sensitive information is protected from unauthorized access. Additionally, the application includes features such as **strong password generation**, structured data storage, and easy retrieval of credentials when required.

The project follows a structured development approach, including **problem analysis, system design, implementation, and testing**. The system architecture is divided into multiple modules, such as credential storage, password generation, encryption handling, and data retrieval. These modules work together to ensure smooth operation and maintain data security.

From a technical perspective, the project utilizes Python and its standard libraries, including **hashlib for encryption, json for structured data storage, random for password generation, and os for file handling**. These tools enable efficient data processing and secure handling of sensitive information.

During the development process, several challenges were encountered, particularly in understanding encryption techniques and designing a reliable storage structure. These challenges were addressed through research, experimentation, and iterative improvements to the system design. The final outcome is a functional Password Manager that meets the basic requirements of security, usability, and performance.

This internship provided valuable learning opportunities in multiple areas, including **Python programming, data security, software design, and problem-solving**. It also enhanced the ability to convert theoretical knowledge into practical applications. The experience gained during this project will be beneficial for future work in fields such as **software development, cybersecurity, and data-driven applications**.

In conclusion, the Password Manager project successfully demonstrates the application of programming and security concepts to solve a real-world problem. It highlights the importance of secure data handling and provides a foundation for developing more advanced systems in the future.

TABLE OF CONTENTS

1 Preface 4

2 Introduction 5

2.1 About UniConverge Technologies Pvt Ltd 5

2.2 About upskill Campus 10

2.3 The IOT Academy 12

2.4 Objective 12

2.5 Reference 13

2.6 Glossary....13

3 Problem Statement 14

4 Existing and Proposed solution 15

5 Proposed Design/ Model 16

5.1 High Level Diagram (if applicable) 16

5.2 Low Level Diagram (if applicable) 17

5.3 Interfaces (if applicable) 18

6 Performance Test 19

6.1 Test Plan/ Test Cases 19

6.2 Test Procedure 19

6.3 Performance Outcome 20

7 My learnings....21

8 Future work scope 22

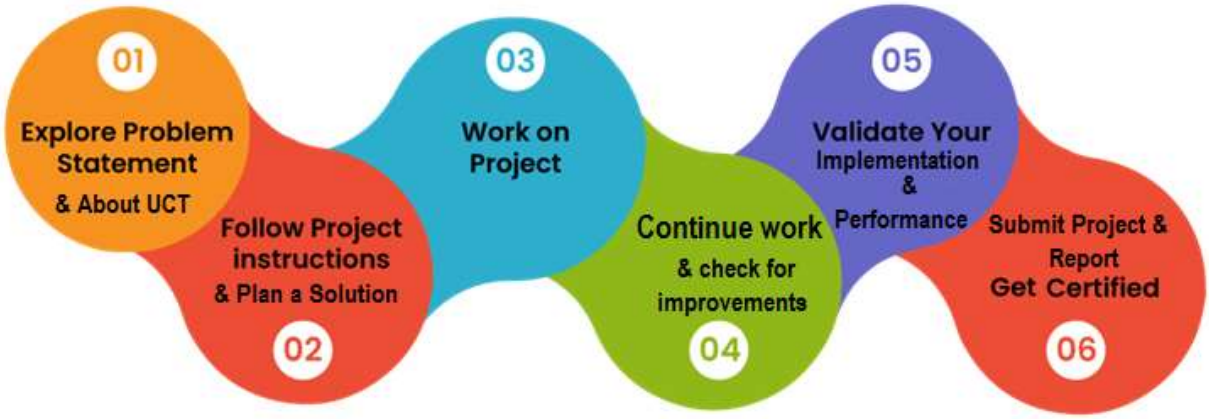
9 Conclusion 22

1 Preface

This internship was designed to provide practical exposure to real-world problems and their solutions using programming skills.

The Password Manager project was chosen to address the growing need for secure password storage. Throughout the internship, various stages such as **problem analysis, design, implementation, and testing** were carried out.

How Program was planned



This project enhanced my knowledge in **Python, file handling, and basic cybersecurity concepts**. I would like to thank upskill Campus, The IoT Academy, and mentors for their guidance and support.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital , providing industrial solutions using technologies such as IoT, Machine Learning, Cloud Computing, and Cybersecurity. It focuses on delivering scalable and efficient solutions for industries.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

i. UCT IoT Platform (UCT Insight)

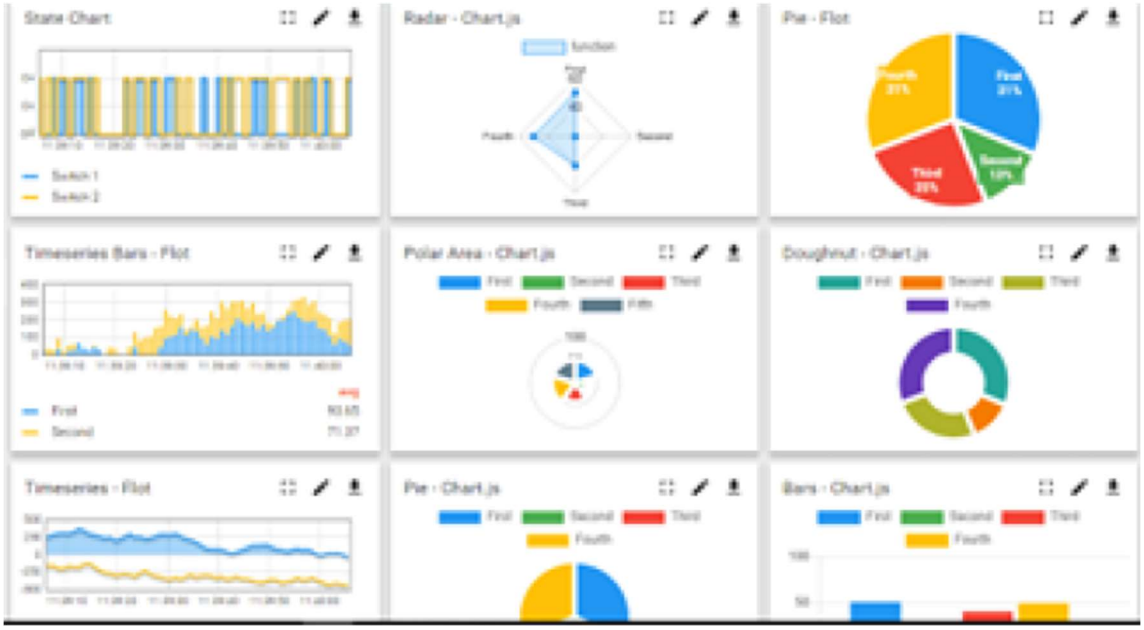
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

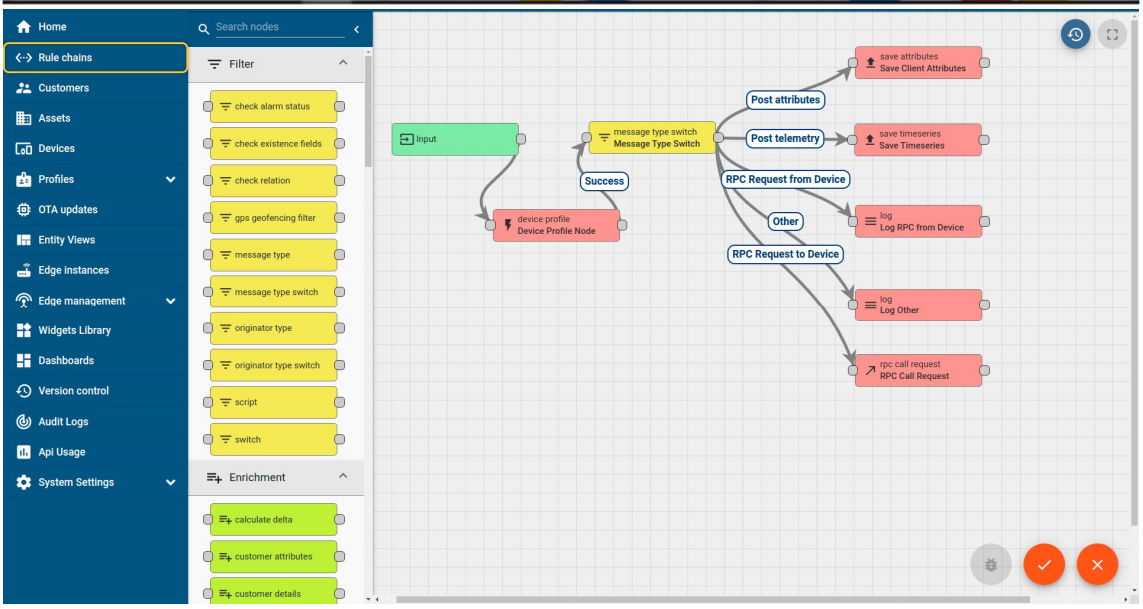
- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

- It supports both cloud and on-premises deployments.

It has features to

• Build Your own dashboard
• Analytics and Reporting
• Alert and Notification
• Integration with third party application(Power BI, SAP, ERP)
• Rule Engine


Stane Chart
Radar - Chart.js
Pie - Flot
Timeseries Bars - Flot
Polar Area - Chart.js
Doughnut - Chart.js
Timeseries - Flot
Pie - Chart.js
Bars - Chart.js


Home
Rule chains
Customers
Assets
Devices
Profiles
OTA updates
Entity Views
Edge instances
Edge management
Widgets Library
Dashboards
Version control
Audit Logs
Api Usage
System Settings
Search nodes
Filter
check alarm status
check existence fields
check relation
gps geofencing filter
message type
message type switch
originator type
originator type switch
script
switch
Enrichment
calculate delta
customer attributes
customer details
Input
message type switch
Message Type Switch
Success
device profile
Device Profile Node
Post attributes
Post telemetry
RPC Request from Device
Other
RPC Request to Device
save attributes
Save Client Attributes
save timeseries
Save Timeseries
log
Log RPC from Device
log
Log Other
rpc call request
RPC Call Request

ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

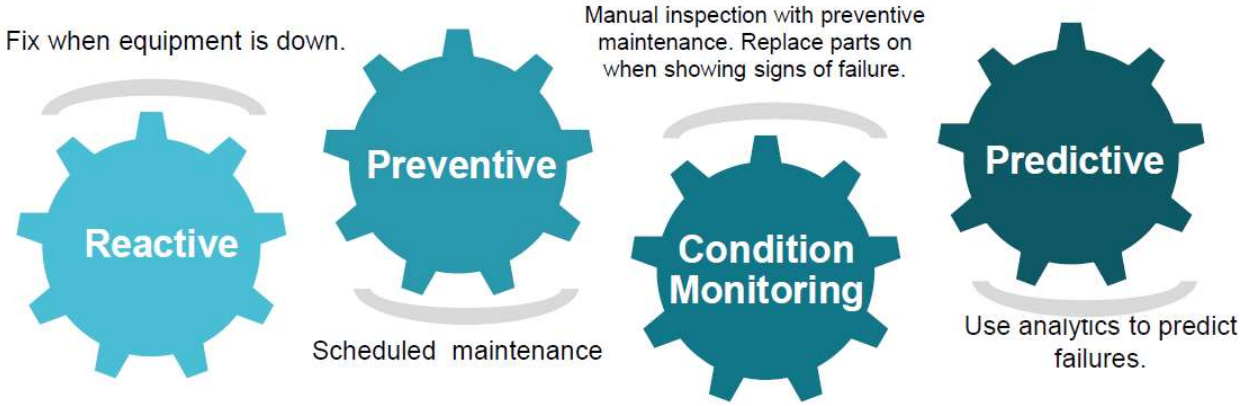
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

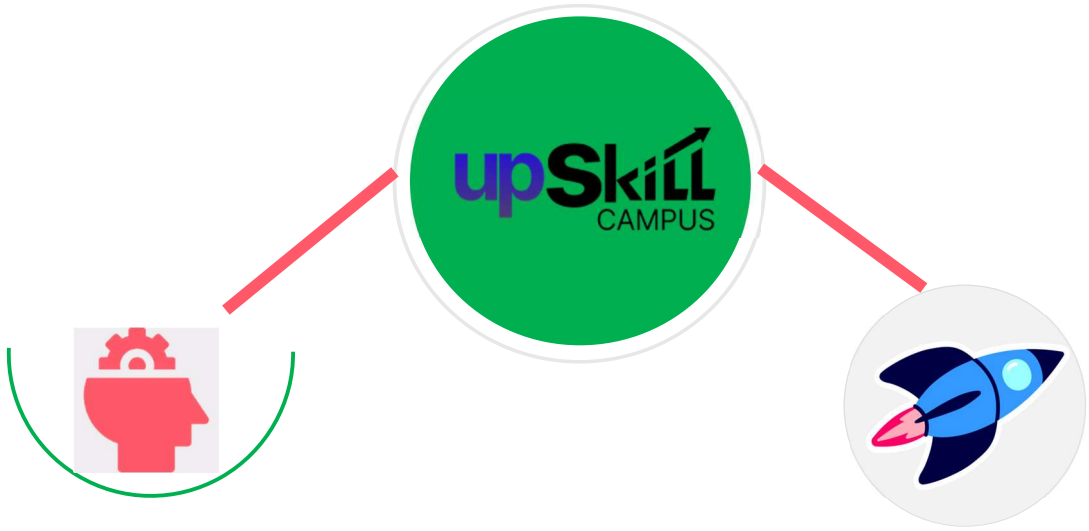
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

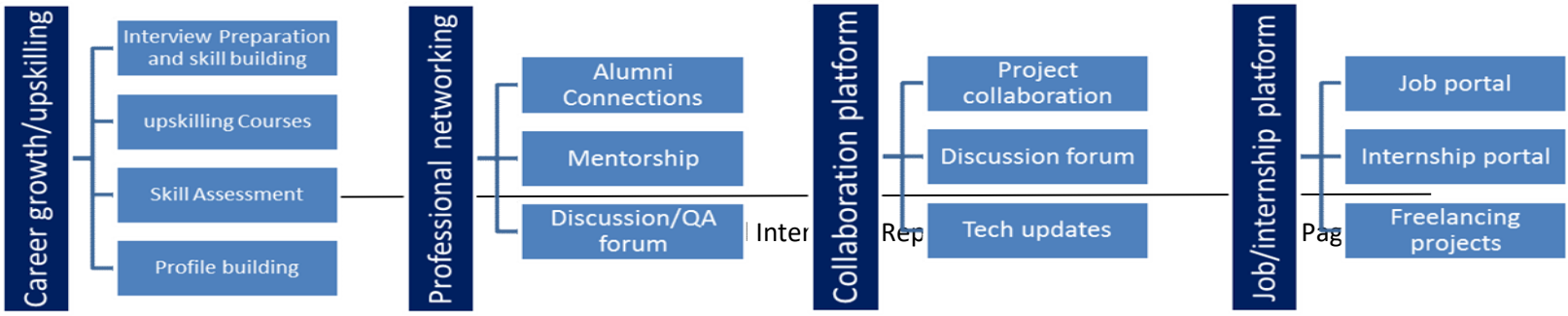
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.
- ☛ to develop a secure password management system
- ☛ to understand real-world software development
- ☛ to apply Python programming in practical scenarios
- ☛ to enhance problem-solving and analytical skills

2.5 Reference

- Python Official Documentation
- IoT Academy Learning Materials
- Online tutorials and articles on password security
- Cybersecurity best practices

2.6 Glossary

Terms	Acronym
Encryption	Process of securing data
Hashing	Converting data into fixed-length code
JSON	Format for storing structured data
CLI	Command Line Interface

3 Problem Statement

Managing multiple passwords manually is difficult and insecure. Users often reuse passwords or store them in unsafe ways, increasing the risk of data breaches.

The problem is to design a **Password Manager system** that:

- Stores passwords securely
- Prevents unauthorized access
- Allows easy retrieval
- Ensures data privacy

4 Existing and Proposed solution

Existing Solutions

- Manual password storage (unsafe)
- Browser-based password saving
- Use of weak or repeated passwords

Limitations

- Lack of proper security
- Risk of data theft
- No centralized secure system

Proposed Solution

The proposed solution is a **Python-based Password Manager** that:

- Stores passwords securely using hashing
- Allows retrieval of stored credentials
- Generates strong passwords
- Provides organized password management

4.1 Code submission (Github link) :

https://github.com/25cs249-create/upskillCampus/blob/main/password_manager.py

4.2 Report submission (Github link) :

https://github.com/25cs249-create/upskillCampus/blob/main

/RAJADITYA_PasswordManager_InternshipReport_USC_UCT.pdf

5 Proposed Design/ Model

The design of this Password Manager is built on a local-first, zero-knowledge security model. This means that *only* the user, possessing the Master Password, has the capability to decrypt the stored credentials. The Python application facilitates management and encryption, but the critical cryptographic keys exist only in volatile memory when the application is active and unlocked. The persistent storage consists entirely of opaque, authenticated ciphertext blobs.

5.1 High Level Diagram

The system is organized into three distinct tiers: the **User Interface (Python/GUI)**, the **Application Core**, and the **Secure Storage**.

- **GUI Tier:** This tier handles all user interaction, built using a standard Python library like Tkinter or PyQt. It manages inputs (like new credentials or the Master Password) and displays retrieved data. It never directly interacts with the database.
- **Application Core:** This is the brains of the operation. It separates business logic from cryptographic functions. When the GUI requests an action (e.g., 'Add Password'), the **Credential Management** module organizes the data (typically as JSON), generates a high-entropy salt, and passes it to the **Encryption/Decryption Engine**.
- **Secure Tier:** This tier is an encrypted SQLite database. Communication with the core is strictly secured. It only accepts or provides encrypted data blobs. The data is unusable without the key derived from the Master Password.

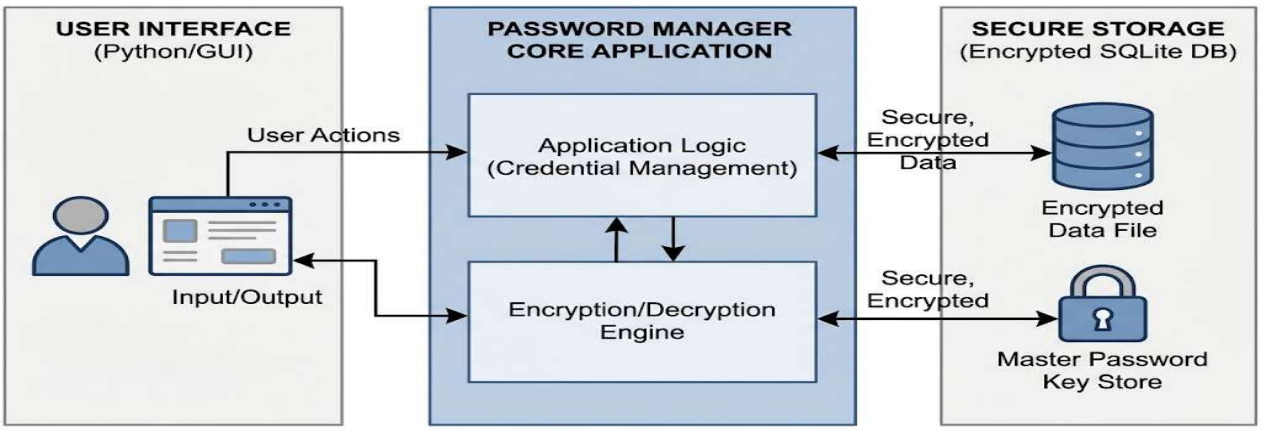


Figure 1: High Level Diagram of the System

5.2 Low Level Diagram

While the HLD provides an overview, a deeper analysis of the most critical security interfaces is required. The Low-Level Diagram zooms in on the **Encryption & Storage Engine** shown in Figure 1, specifically illustrating how a new credential (a standard JSON block) is transformed into an encrypted, authenticated blob for SQLite storage. This LLD details the granular cryptography process.

- **Inputs:** The diagram shows two distinct inputs: 'Plaintext Credentials (JSON)' and the 'User Master Password'.
- **Key Derivation:** The Master Password is not used directly. To ensure security, it is fed into a **PBKDF2 (Password-Based Key Derivation Function 2)** module, which uses a random salt to stretch the password into a cryptographically strong 256-bit derived key.
- **AES-256 GCM MODULE:** This is the core engine. A random 96-bit Initialization Vector (IV) is generated. The derived key and plaintext are then processed using **AES-256 GCM (Galois/Counter Mode)**, a standard authenticated encryption protocol. This is critical: GCM provides *confidentiality* (no one can read the data) and *authenticity* (an Authentication Tag is generated, meaning the data cannot be modified in the database).
- **Outputs:** The diagram details how the multiple cryptographic parts (IV, Auth Tag, Ciphertext) are **assembled into a specific data structure** (a BLOB) and successfully written to the database cylinder (reusing the visual style of the DB from Figure 1). The database stores only this combined binary blob.

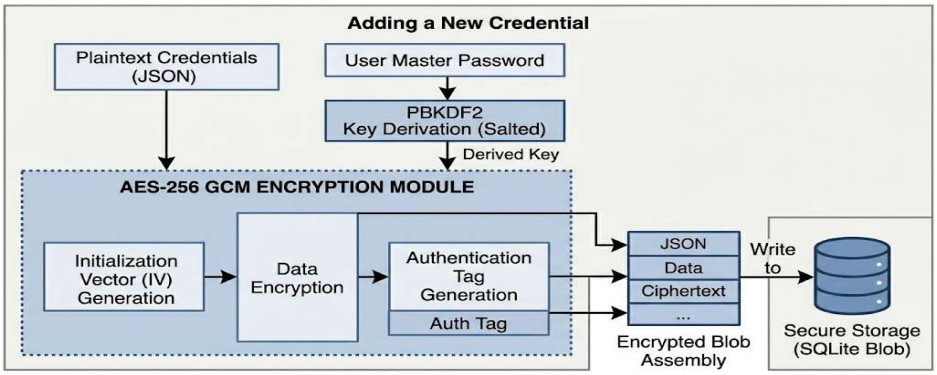


Figure 2: Low Level Diagram - Encryption & Storage Engine

5.3 Interfaces (if applicable)

The design requirements emphasize how different functional interfaces must interact securely. Figure 3 combines two crucial perspectives: a **Functional Data Flow Diagram (DFD)** and a **State Machine**, along with the defined **Data Structure & Protocols** necessary for communication between modules.

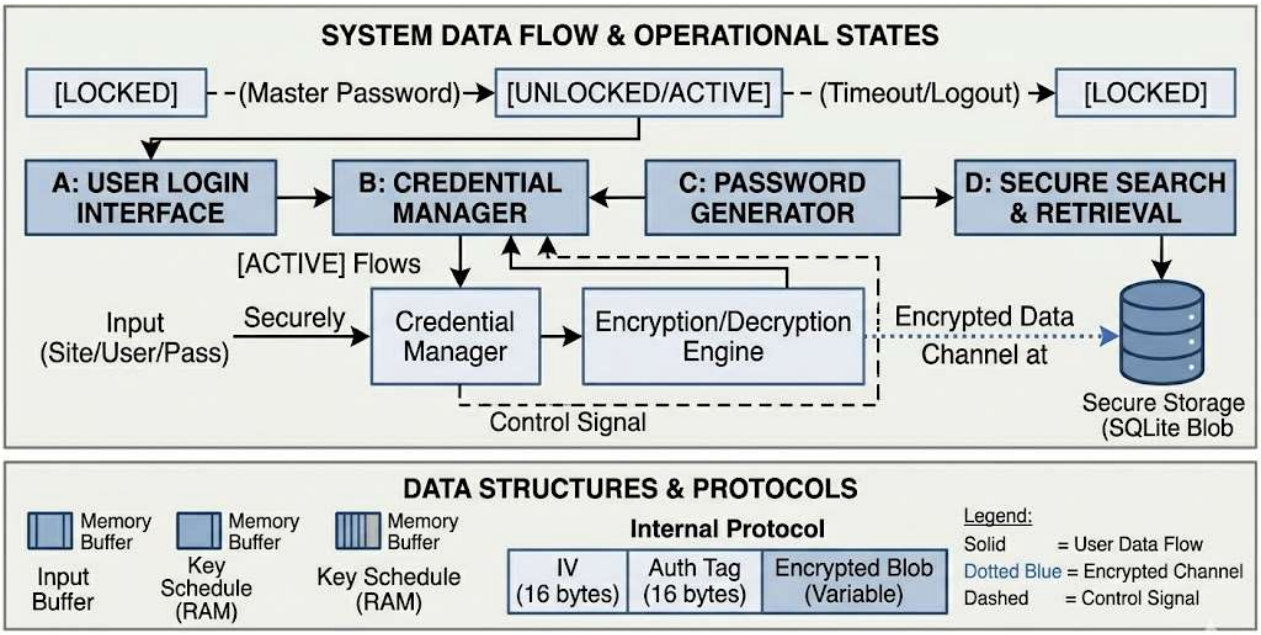


Figure 3: DATA STRUCTURES & PROTOCOLS - INFORMATION EXCHANGE

6 Performance Test

6.1 Test Plan / Test Cases

The Password Manager was tested for the following cases:

1. **Add new password**
    - Input valid website, username, and password.
    - Expected result: Data stored successfully.
2. **Generate strong password**
    - Request password generation.
    - Expected result: Random secure password is generated.
3. **Retrieve existing password**
    - Search for a saved website.
    - Expected result: Correct password details are displayed.
4. **Handle invalid input**
    - Provide incomplete or incorrect data.
    - Expected result: Program shows error message or validation warning.

6.2 Test Procedure

- Run the Python program.
- Select the required menu option.
- Provide test input data.
- Observe the output and verify correctness.
- Repeat the process for different test cases to ensure reliability.

6.3 Performance Outcome

The application performed well in basic testing:

- password storage worked correctly,
- password generation produced secure random values,
- retrieval was fast and organized,
- the file-based structure handled data efficiently.

The system met the main requirements of secure storage and easy password management.

7 My learnings

This internship project helped me learn several important skills:

- how to design a practical Python application,
- how to structure a program into modules,
- how to use file handling for storing data,
- how to use hashing/encryption concepts for security,
- how to generate strong passwords,
- how to test and improve a project systematically.

I also learned the importance of secure coding and why password protection is essential in real-world software. This project improved my problem-solving ability, programming confidence, and understanding of how a simple application can solve a common user problem.

8 Future work scope

The Password Manager project can be improved further in the future through:

- **Graphical User Interface (GUI)** using Tkinter,
- **Database integration** using SQLite or MySQL,
- **Master password authentication** for improved security,
- **Cloud synchronization** for multi-device access,
- **Advanced encryption methods** for stronger protection,
- **Password strength analysis** before saving credentials,
- **Password backup and restore** features.

These enhancements would make the application more practical, secure, and user-friendly.

9 Conclusion

The Password Manager project successfully demonstrates how Python can be used to build a secure and useful real-world application. It addresses the common problem of password management by offering safe storage, retrieval, and generation features. The project strengthened my technical knowledge and provided valuable experience in software design, security concepts, and application development.